

Angry Agents

A Multi-Agent Persona Simulation and Evaluation Platform

AI Design 2026 — Final Project Report

May 2026

Team:

Alfonso Antognozzi Davide Cabitza Gabriele Fronzoni
Kevin Shimaj Marco Massa



Contents

1	Introduction	2
1.1	Interaction modes	2
1.2	Persona types	2
1.3	User roles	2
2	Project Evolution: From Commits to Architecture	2
2.1	Phase I — Foundation (May 5–13)	2
2.2	Phase II — Core Pipeline (May 14–24)	3
2.3	Phase III — Integration (May 25–27)	4
2.4	Phase IV — Polish and MCP (May 28–31)	6
3	System Architecture	6
3.1	Technology stack	6
3.2	Layered architecture	6
3.3	Database schema	7
4	Persona Extraction and Profiles	7
4.1	Source material	7
4.2	Two-phase extraction pipeline	7
4.3	Structured profile schema	8
5	Context Management	8
5.1	Token budget per turn	9
5.2	Mitigation mechanisms	9
6	Chat Simulation	9
6.1	PersonaAgent	9
6.2	TurnScheduler	10
6.3	GroupChatSession	10
6.4	DM chat	10
7	Evaluation Framework	10
7.1	Four fidelity dimensions	10
7.2	Phase 1 — Independent evaluation	10
7.3	Statistical metrics	11
7.4	Phase 2 — Deliberation (design)	11
8	MCP Layer	12
8.1	Architecture	12
8.2	Tool tiers	12
9	Knowledge Graph	12
10	State-of-the-Art Positioning	13
11	Open Questions and Future Work	13
12	References	14

Introduction

Angry Agents is a web application in which users interact with AI persona-agents that faithfully represent real-world or fictional characters — politicians, influencers, movie characters, podcast hosts, and more. Conversations are automatically evaluated by 20 judge-agents across four fidelity dimensions: persona identification, individual fidelity, group fidelity, and behavioral fidelity.

The project addresses a research question at the intersection of multi-agent systems, LLM-based role-playing, and automated evaluation:

Can an LLM, conditioned on a structured persona profile extracted from source material, faithfully simulate a real or fictional character in a multi-turn conversation, to the point that independent judge-agents can correctly identify it?

Interaction modes

The platform supports two conversation types:

DM One user converses one-on-one with a single persona-agent. The agent responds in character after every user message.

Group chat The user sets a topic and selects 2–8 agents. The agents converse autonomously in the background; the user can observe and inject messages.

Persona types

Fiction Characters extracted from movie scripts, TV screenplays, and books (e.g. Batman, Darth Vader, Bart Simpson).

Real-world People extracted from podcast transcripts, YouTube videos, and social-media corpora (e.g. Barack Obama, Andrej Karpathy, Bernie Sanders).

User roles

Common user Chat with agents, manage own conversations.

Admin Manage personas, launch judge evaluations, view metrics.

Project Evolution: From Commits to Architecture

The repository contains **308 commits** spanning from May 5 to May 31, 2026, authored by five contributors. The commit history reveals four distinct development phases.

Phase I — Foundation (May 5–13)

Problem definition and SOTA. The project opened with iterative refinement of the problem statement (`NEW_PROBLEM.md`) and a five-paper literature review (`SOTA.md`) covering: Wason et al. [2024] on dialogue agent failure modes; Salewski et al. [2023] on in-context impersonation and demographic bias; Jiang et al. [2024] (PersonaLLM) on

Phase	Dates	Peak activity
I — Foundation	May 5–13	May 11 (21 commits), May 13 (11 commits)
II — Core pipeline	May 14–24	May 18 (28), May 20 (27), May 24 (26)
III — Integration	May 25–27	May 27 (38 commits — busiest day)
IV — Polish & MCP	May 28–31	May 28–31 (12–16/day)

Figure 1: Commit density by phase. May 27 is the single most active day with 38 commits.

Contributor	Commits
Gabriele Fronzoni	98
Kevin Shimaj	79
Alfonso Antognozzi	56
Davide Cabitza	41
Marco Massa	34

Figure 2: Commit count per contributor.

psychometric validation; Zhou et al. [2025] on behavioral fidelity for real-person agents; and Park et al. [2023] (Generative Agents) on memory, reflection, and group dynamics.

Key insight from the review: *personal detail features are the single most critical driver of behavioral fidelity* (Zhou et al., 2025), motivating the structured profile approach. Park et al. identified excessive agreeableness as a failure mode in group simulations — directly motivating the perturbation mechanism.

Database design v1. Commit ba58069 introduced the first schema (DB_v1.md), with six tables: `Agents`, `Chat`, `Chat_messages`, `Judges`, `Judge_evaluation`, and two context tables (`Podcast_Agent_Context`, `Fiction_Agents_Context`).

The most important privacy design decision appeared here: the `author` field in `Chat_messages` stores `name + surname + DIGEST` — a cryptographic hash preventing judges from linking messages back to known agent identities. Crucially, **no foreign key** from `Chat_messages` to `Agents` was added; the anonymisation is enforced at the schema level.

Scraping and first personas. Three scraping pipelines were bootstrapped: YouTube transcript extraction (4541c55), movie-script PDF parsing (ba4ec24), and IMSDB scraping. The first batch of fictional personas was committed: Darth Vader, Yoda, Indiana Jones, Jack Sparrow, Luke Skywalker, Riggan, Batman, Joker, Po, Rocky, Tony Stark (11 profiles).

First UI prototype. Commit 7a822d6 landed the first React/JSX UI with ten components covering auth, home, library, DM and group chat wizard, active chat, and admin dashboard — all wired to static mock data (`data.js`).

Phase II — Core Pipeline (May 14–24)

Database v2 and full REST API. The schema was revised to DB_v2.md: the two separate context tables were unified into `Agent_context` (storing arbitrary `signature_phrases`

as JSON), a `Topic` table was added, and all entities received `created_at` / `updated_at` / `deleted_at` timestamps with server-side triggers. The `Chat` table was renamed to `Group_chat` and gained `ID_topic`.

Alfonso Antognozzi built the complete persistence stack: raw-SQL CRUD repositories for all eight entities, a service layer with business logic (HMAC author token, 20-evaluation cap per chat, PBKDF2-SHA256 passwords with 260 000 iterations), and a FastAPI application with nine route files, Pydantic schemas, and Swagger UI.

JWT authentication (commits 7a011e8, 63d13d2). A full stateless auth system was added: HS256 access tokens (15-min expiry), httpOnly refresh tokens (7-day, SHA-256 hash stored in `RefreshToken` table), and `AUTH_DISABLED=1` dev mode. The `require_admin` dependency enforces role-based access.

Database v3 and User table. `DB_v3.md` (May 19) introduced the `User` table with roles (common / admin) and propagated a `Created_by` foreign key to four tables: `Group_chat`, `Topic`, `Agents`, `Chat_messages`. The `Judge_evaluation` composite primary key (`ID_judge`, `ID_chat`) was formalised.

Judge architecture. Davide Cabitza implemented the judge class hierarchy (`BaseJudge` with four abstract evaluation methods) and three concrete specialisations:

- **StyleJudge** — vocabulary, sentence structure, rhetorical habits.
- **IdeologyJudge** — political views, moral stances, belief systems.
- **GeneralJudge** — all observable traits combined.

Jinja2 templates (`.j2` files) extracted prompts from Python, enforcing motivation-before-rating, explicit 1–5 rubric, and JSON response format.

PersonaAgent and GroupChatSession. The `PersonaAgent` class was introduced, wrapping a DB agent row into a live chat-ready object with `dominance_weight` (drives turn scheduling), `cooldown_turns` (derived from `core_style.rhythm`), and `burst_size` (derived from `core_style.sentence_shape`).

The `GroupChatSession` orchestrator was paired with a `TurnScheduler` (`round_robin` or `weighted_random`) and a `ContextWindow` (rolling or selective trimming of history).

The `deviation()` logging convention was introduced (`STRICT_MODE=1` turns warnings into hard raises, enabling strict CI).

Test suite. By end of Phase II the test suite covered 291 tests across all eight repositories and services. Tests include boundary cases for soft-delete, composite PKs, the 20-evaluation cap, password hashing, JWT lifecycle, and slug collision.

Phase III — Integration (May 25–27)

May 27 produced the highest single-day commit count (38), reflecting the convergence of multiple parallel workstreams.

RAG evaluation pipeline (Alfonso Antognozzi). The entire `src/rag/` package was built from scratch:

- **builder.py** — splits each persona profile into typed semantic chunks (`style`, `voice`, `worldview`, `behavior`, `quote`) with configurable field weights per judge role.
- **indexer.py** — embeds chunks using `text-embedding-3-small` and stores them in ChromaDB under `data/chroma/`.
- **retriever.py** — queries ChromaDB, applies per-role field weights, runs Maximal Marginal Relevance (MMR) re-ranking ($\lambda = 0.7$), and returns the top-20 shortlist for each judge.

This reduced judge LLM calls from $\sim 2\,000$ to ~ 80 (-97%) and latency from 33 minutes to ~ 30 seconds for a full evaluation run.

Hungarian algorithm for persona identification. The greedy argmax had a critical flaw: a single high-scoring author could be assigned to multiple personas simultaneously. The fix replaced argmax with `scipy.optimize.linear_sum_assignment` (Hungarian algorithm), enforcing a bijective author $\leftrightarrow$ persona mapping.

Two-phase persona extraction (Gabriele Fronzoni). `extract_profile.py` was rewritten to make two separate LLM calls per persona:

- Phase 1** `gpt-4o-mini` with the full transcript — grounds all behavioral fields (`core_style`, `worldview`, `emotional_tells`, etc.) in real observed speech.
- Phase 2** `gpt-4o` with *no transcript* — extracts pure model-knowledge fields (`favored_words`, `structural_patterns`, `do_not_say`) that source transcripts alone under-specify. Each Phase 2 entry must pass a “similar figure” discriminability test.

Total extraction cost for 80 personas: **\$0.57** (1.19M tokens).

Full UI wiring (Kevin Shimaj). All static mock data in `data.js` was replaced with live API calls. New `api.js` and `agents-store.jsx` were added. The admin dashboard was wired to render `metrics_report.json` output — accuracy, confusion matrix, fidelity scores, Gini coefficient.

MCP layer (Kevin Shimaj). The Model Context Protocol server was implemented at `src/mcp/`, exposing the platform to any MCP-compatible LLM client (Claude Code, Claude Desktop). Two tiers of tools:

- Tier 1** Read-only tools (no confirmation): `get_agents`, `get_chat_messages`, `get_judge_result`, etc.
- Tier 2** Write tools (preview $\rightarrow$ explicit user confirmation $\rightarrow$ execute): `confirm_create_full_chat`, `confirm_start_judging`, etc.

Evaluation metrics. The statistical evaluation engine (`src/eval/`) was completed with four metric modules: `metrics_persona_id.py`, `metrics_fidelity.py`, `metrics_group.py`, `metrics_deliberation.py`, all backed by a shared non-parametric `bootstrap_ci()` helper (10 000 resamples).

Phase IV — Polish and MCP (May 28–31)

The final phase focused on UI polish (white theme, responsive layout, info chats), MCP refinements (writing tools, reading tools, ui-routes updates), and evaluation improvements:

- MAD (Mean Absolute Deviation) between all judge types added to admin dashboard.
- Occupation field added to real-world persona profiles and wired to the agent library subtitle.
- Batch evaluation pipeline (EVAL_BATCH.md): aggregates per-chat `metrics_report.json` files using bootstrap CI for $n < 20$ chats and CLT (normal approximation) for $n \geq 20$.
- Knowledge graph (graphify-out/graph.json) committed — 6 920 nodes, 8 337 edges, covering 20 community clusters.
- Filters for chat list added; pause button for group chat; last icons.

System Architecture

Technology stack

Layer	Technology
Backend API	Python 3.12, FastAPI, Uvicorn
Database	SQLite (raw SQL, WAL mode, FKs enforced)
Schema/validation	Pydantic v2
Auth	JWT (HS256), PBKDF2-SHA256, httpOnly cookies
LLM backend	OpenAI (gpt-4o-mini) or Ollama (mistral)
RAG	ChromaDB, <code>text-embedding-3-small</code>
Frontend	Vanilla React/JSX, no bundler
Testing	pytest, in-memory SQLite fixtures
MCP	FastMCP (<code>src/mcp/mcp_server.py</code>)

Layered architecture

The backend follows a strict layered pattern:

1. **Routes** (`src/API/routes/`) — HTTP layer only. Receives requests, calls services, returns responses. Never touches SQL directly.
2. **Services** (`src/db/services/`) — Business logic. Calls repositories, enforces invariants (e.g. 20-evaluation cap, slug uniqueness).
3. **Repositories** (`src/db/repositories/`) — Raw SQL CRUD. Every entity exposes: `create`, `get`, `get_by_slug`, `update`, `delete(hard=False)`, `query(filters, limit, offset)`.
4. **Models** (`src/db/models/`) — Python dataclasses mirroring DB tables.

Key invariant: SQL lives only in `api/`. Agent and evaluation code receives plain Python objects.

Database schema

The final schema (v3, implemented in `src/db/models/base.py`) comprises ten tables:

Table	Purpose	Notable constraints
User	Platform users	Role: common admin; slug UNIQUE
Agents	Persona profiles	Summary: JSON profile; Created_by FK
Agent_context	Context chunks	Signature_phrases: JSON phrases
Topic	Chat themes	Title UNIQUE
Group_chat	Chat sessions	status: pending running done stopped error
Chat_agent	Chat↔Agent junction	Composite PK (id_chat, id_agent)
Chat_messages	Messages	author: name+surname+DIGEST; no FK to Agents
Judges	Judge instances	Role: style ideology general behavioral
Judge_evaluation	Evaluation scores	Composite PK; max 20 per chat enforced in service
RefreshToken	JWT refresh tokens	Stores SHA-256 hash, never plaintext

Soft-delete pattern. All tables have `deleted_at`. Queries always filter `WHERE deleted_at IS NULL`. Hard delete is admin-only. Slug uniqueness is `UNIQUE WHERE deleted_at IS NULL`.

Author anonymisation. `Chat_messages.author = HMAC-SHA256(ANGRY_AUTHOR_SECRET, name:surname)`. The hash is computed server-side by `ChatMessageService`. No foreign key from `Chat_messages` to `Agents` exists — judges can distinguish different message sources without knowing which agent sent what.

Persona Extraction and Profiles

Source material

- **Fiction personas:** movie scripts (IMSDb HTML, PDFs), TV screenplays. Challenge: sparse material — a character may have only a few dozen lines.
- **Real-world personas:** YouTube transcripts, podcast audio, Twitter/X threads. Challenge: extracting ideology and behavior from naturalistic speech rather than written dialogue.

Target source material: **10 000–50 000 tokens per persona** to capture not just surface vocabulary but deeper patterns: argumentation style, emotional triggers, social positioning.

Two-phase extraction pipeline

The extraction pipeline (`src/agents/personas/extract_profile.py`) was redesigned in Phase III to use two separate LLM calls:

Phase 1 — Transcript-grounded fields. Uses `gpt-4o-mini` with the full source transcript. Extracts behavioral fields that require grounding in real observed speech: `core_style`, `worldview`, `self_image_vs_reality`, `emotional_tells`, `situational_behavior`, `escalation_pattern`, `relationship_matrix`, `annotated_quotes`.

Phase 2 — Model-knowledge fields. Uses `gpt-4o` with *no transcript*. Extracts fields the model knows from parametric knowledge: `vocabulary_fingerprint.favored_words`, `speech_signature.structural_patterns`, `do_not_say`. Each entry must fail a “similar figure” discriminability test (i.e. it cannot be equally attributed to a confusable persona). Total cost for 80 personas: \$0.57 (1.19M tokens).

Structured profile schema

Each profile is a JSON document stored in `Agents.Summary`. Key fields:

Field	Content
<code>core_style</code>	Default register, sentence shape, rhythm, two registers
<code>humor</code>	Style, frequency, mechanism
<code>speech_signature</code>	Naming behavior, structural patterns
<code>vocabulary_fingerprint</code>	Favored words, avoided words, domain jargon, fillers
<code>worldview</code>	Key beliefs/values as key-value pairs
<code>self_image_vs_reality</code>	Self-image, reality, gap behavior
<code>emotional_tells</code>	When guarded, afraid, grieving, in control
<code>situational_behavior</code>	When outmatched, disagreeing, protecting, proven wrong
<code>escalation_pattern</code>	How the character escalates conflict
<code>conversation_goals</code>	List of what the character tries to achieve
<code>relationship_matrix</code>	Per-character relationship descriptions
<code>knowledge_domains</code>	Expert / surface / ignorant lists
<code>social_positioning</code>	Desired position, actual dynamic, contradiction
<code>annotated_quotes</code>	Quote + context + what it illustrates
<code>do_not_say</code>	Lines that would break character, with reason

Context Management

Long multi-turn conversations present three distinct problems for persona fidelity:

1. **Grounding** — the model has no inherent knowledge of the persona and must receive its profile on every inference call.
2. **Character drift** — after ~30–40 turns, the conversational context dominates the system prompt and the agent converges toward a generic LLM voice.
3. **Group dynamic drift** — in group chats with 8 agents, they converge toward similar tone and positions (excessive agreeableness, Park et al., 2023).

Token budget per turn

Component	Tokens (approx.)
Structured profile (cached)	~1 000
RAG chunks (dynamic)	~400
Compressed history (dynamic)	~300
Self-anchor / reflection	~200 (savings $\approx 92\%$)
Last 5 turns verbatim	~600
User message	~50
Total per turn	~2 550
<i>vs. raw transcript</i>	<i>~30 000</i>

Mitigation mechanisms

Sliding window + compression. Only the last $K = 5$ turns are passed verbatim. Every 10 turns, the older history is compressed into ~200–300 tokens via an LLM call and stored as a summary.

Reflection. Every $N = 15\text{--}20$ turns, the agent reviews its last messages against its profile and produces a 150–200-token self-anchor injected before the next reply.

Perturbation (group chat only). Every 10–15 turns per agent, a private signal is injected: “The conversation is moving toward consensus. React in a way that reflects your genuine position.” This fires on a per-agent basis and is not visible to other agents.

Drift detection. Pairwise cosine similarity between agents’ last-turn message embeddings is tracked. A rising similarity above threshold 0.85 triggers early reflection or perturbation.

Prompt caching. The structured profile is passed with a cache breakpoint after the system prompt (Anthropic ephemeral cache, 5-min TTL), saving 60–80% of input token cost on the static portion after the first turn.

Chat Simulation

PersonaAgent

`PersonaAgent` (`src/agents/personas/persona_agent.py`) is the core runtime class for a single persona in a chat:

- `bind_to_chat(chat_id, topic)` — resolves profile from DB, derives `dominance_weight` from `social_positioning` keywords, `cooldown_turns` from `core_style.rhythm`, `burst_size` from `core_style.sentence_shape`.
- `respond(history, turn_count)` — renders the Jinja2 prompt template, calls the LLM, returns the reply string.

TurnScheduler

Selects the next speaking agent:

- **round_robin** — cycles through agents in fixed order.
- **weighted_random** — samples proportional to `_effective_weight`, which penalises an agent within its `cooldown_turns` window by factor 0.1.

GroupChatSession

Orchestrates the full autonomous conversation loop:

1. Select next agent via `TurnScheduler`.
2. Trim history via `ContextWindow` (`rolling` or `selective`).
3. Call `agent.respond()`.
4. Persist message with HMAC author token via `ChatMessageService`.
5. Repeat `burst_size` times, reading fresh history between bursts.
6. Check `Group_chat.status` at each iteration — stop if `stopped`.

DM chat

`DMSession` (`src/dm_chat/session.py`) is a simplified single-agent path with no scheduler, no burst, no background task. Agent responds synchronously after each user message.

Evaluation Framework

The evaluation pipeline operates in two phases and measures four fidelity dimensions.

Four fidelity dimensions

Persona ID	Can judges correctly identify which anonymous author tag corresponds to which persona? Baseline: 12.5% (random over 8).
Individual	How faithfully did each agent represent its persona? Scored 1–5 per judge.
Group	Does the conversation exhibit realistic group dynamics (turn distribution, topic diversity)?
Behavioral	Does the conversation read as human-produced, or does it show LLM artifacts (excessive politeness, unnatural consensus)?

Phase 1 — Independent evaluation

All 20 judges evaluate independently. They are divided by role:

Role	Focus
style	Vocabulary, sentence structure, rhetorical habits
ideology	Political views, values, moral stances
behavioral	Behavioral fidelity, human-likeness
general	All traits combined

RAG pre-filtering. Before scoring, each judge queries ChromaDB to receive a shortlist of 20 candidate personas (from potentially 100+). Field-weighted retrieval boosts chunks relevant to the judge’s role. MMR re-ranking ensures diversity in the shortlist.

Hungarian assignment. Author $\leftrightarrow$ persona matching uses `scipy.optimize.linear_sum_assignment` to enforce a bijective assignment — each author maps to exactly one persona.

Statistical metrics

Persona identification.

$$\text{accuracy} = \frac{\text{correct guesses}}{\text{total guesses}} \quad (1)$$

$$H_0 : p = \frac{1}{8} = 0.125, \quad H_1 : p > 0.125 \quad (\text{one-sided binomial exact test}) \quad (2)$$

$$\kappa = \frac{p_o - p_e}{1 - p_e} \quad (\text{Cohen’s Kappa, corrects for chance}) \quad (3)$$

Individual fidelity. Median + IQR over 20 Likert ratings per persona. Bootstrap 95% CI on the median (10 000 resamples). Judge-type agreement measured by MAD between median scores per type.

Group fidelity. Gini coefficient on turn distribution:

$$G = \frac{2 \sum_i i \cdot x_i}{n \sum_i x_i} - \frac{n+1}{n}$$

Target range for realistic group chats: $G \in [0.28, 0.42]$ (reference mean 0.33, std 0.05).

Batch evaluation. The batch pipeline (`src/eval/batch.py`) aggregates per-chat metrics across many sessions. CI method: bootstrap for $n < 20$ chats; CLT normal approximation ($\text{CI} = \bar{x} \pm 1.96 \cdot \sigma / \sqrt{n}$) for $n \geq 20$.

Phase 2 — Deliberation (design)

Phase 2 takes high-variance cases from Phase 1 into structured deliberation:

1. Identify cases where judge variance exceeds threshold.
2. Run 3–5 deliberation rounds. Judges explain evidence, revise ratings.
3. Track variance per round; convergence = $\text{var}_{\text{final}} < \text{var}_0$.
4. F-test between round 0 and final round variances.
5. Track confidence calibration: $r(\Delta\text{confidence}, \Delta\text{accuracy})$.

MCP Layer

The Model Context Protocol server (`src/mcp/mcp_server.py`) exposes the platform to any MCP-compatible LLM client via structured tool calls over stdio.

Architecture

```

LLM client (MCP host)
  | tool call (JSON over stdio)
  v
mcp_server.py    [FastMCP]
  |-- tools/read.py    (Tier 1: free reads)
  \-- tools/write.py   (Tier 2: preview -> confirm)
      |
      v
      client.py    [httpx -> FastAPI :8000]
      |
      v
      SQLite DB

```

Tool tiers

Tier 1 — Read (no confirmation required). `get_agents`, `get_agent_by_id`, `get_chat_messages`, `get_admin_overview`, `get_judge_result`, `get_judge_status`, and more. The `get_agents` tool strips the `summary` field (10k–50k tokens per agent) to prevent context overflow; full profiles retrieved by ID.

Tier 2 — Write (preview → confirm). Every write has two tools: `preview_*` shows the payload; `confirm_*` executes only after explicit user approval.

Preview / Confirm pair	Action
<code>create_full_chat</code>	Create DM or group chat
<code>create_message</code>	Post a message to a chat
<code>stop_chat</code>	Halt background agent loop
<code>start_judging</code>	Launch 20-judge evaluation (~\$0.40, ~9 min)
<code>create_topic</code>	Create a new topic

The LLM **never stops a chat autonomously** — only when the user explicitly writes “stop”.

Knowledge Graph

A pre-built knowledge graph of the entire codebase was committed at `graphify-out/graph.json` (6 920 nodes, 8 337 edges, generated via 88 LLM agent calls). The graph is organised into 20 community clusters:

Architecture & Core Concepts	Persona Agent Engine
Application Core & Auth	Persona Profile Extraction
Agent API & Repository	RAG Index Builder
User API & Service	UI & Admin Routes
Group Chat Runtime	UI Component Library
Judge Pipeline	Admin Dashboard UI
Evaluation Statistics Engine	... (20 clusters total)

God nodes (highest-degree abstractions): `UserService` (45 edges), `GroupChatService` (41), `TopicService` (39), `AgentService` (35).

Key architectural decisions captured as hyperedges: the five-mechanism persona context stack (structured profile + RAG + sliding window + reflection + perturbation); the two-phase judge pipeline; the anonymisation bridge between `Chat_messages` and `Agents`.

State-of-the-Art Positioning

Five papers ground the project methodology:

Wason et al. (2024) Identifies context management and response accuracy/dialogue success divergence as core unsolved challenges — directly motivating the sliding window, compression, and reflection mechanisms.

Salewski et al. (2023) Documents demographic bias in persona prompts. Warns that persona prompts specifying demographic characteristics induce stereotyped behavior — motivating the use of behavioral/ideological profiles rather than demographic labels.

Jiang et al. (2024) — **PersonaLLM** Validates that LLMs reliably encode Big Five trait profiles in self-reports and linguistic style. Critically: informing raters of AI authorship degrades personality attribution accuracy — motivating the **blind evaluation protocol** (anonymised author tags).

Zhou et al. (2025) Shows personal detail features are the single most critical driver of behavioral fidelity (removing them causes the largest accuracy drop). Motivates the structured profile schema.

Park et al. (2023) — **Generative Agents** Demonstrates memory, reflection, and planning for long-term coherent agents. Documents excessive agreeableness in group settings — motivating the perturbation mechanism.

Novel contribution. No prior work integrates all four fidelity dimensions (persona identification, individual, group, behavioral) in a unified experimental pipeline grounded in real-person text corpora. The project’s evaluation framework constitutes the novel scientific contribution.

Open Questions and Future Work

Deliberation (Phase 2) The phase 2 pipeline is designed (documented in `EXPERIMENTATION.MD` and `EVAL.md`) but not yet fully implemented. Implementing structured multi-round deliberation and measuring variance reduction would complete the evaluation pipeline.

Behavioral fidelity Unlike persona identification (clean baseline) and individual fidelity (Likert scores), behavioral fidelity — whether an agent reads as human-produced — lacks an operational definition. LIWC-based psycholinguistic profiling or human blind raters are the most viable approaches.

Optimal conversation length Too short (<20 turns) misses deep behavioral patterns; too long (>150 turns) causes drift. The empirically optimal range is not yet determined.

Candidate set size for identification The evaluation uses a closed 8-tag set (clean baseline, 12.5% random). An expanded-set design ($K > 8$ decoy personas) would be more ecologically valid but reduces statistical power. Three options are analyzed in `evaluation_design_open_questions.md`.

RAG recall for sparse personas Retrieval recall degrades for personas with <5k tokens of source material. No mitigation is implemented for sparse profiles.

MCP auth MCP read/write tools currently bypass FastAPI auth. The `Authorization` header injection needs to be added before production use.

Prefiltering experiments Varying candidate set size for judge shortlisting and measuring the effect on final accuracy would quantify the RAG retrieval quality vs. judge workload trade-off.

References

Jiang, H., Zhang, X., Cao, X., Breazeal, C., Roy, D., & Kabbara, J. (2024). **PersonaLLM: Investigating the Ability of Large Language Models to Express Personality Traits**. *Proceedings of ACL 2024*. arXiv:2305.02547.

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). **Generative Agents: Interactive Simulacra of Human Behavior**. *Proceedings of UIST '23*. ACM. <https://doi.org/10.1145/3586183.3606763>

Salewski, L., Alaniz, S., Rio-Torto, I., Schulz, E., & Akata, Z. (2023). **In-Context Impersonation Reveals Large Language Models' Strengths and Biases**. *NeurIPS 2023*. arXiv:2305.14930.

Wason, R., Arora, P., Arora, D., Kaur, J., Singh, S. P., & Hoda, M. N. (2024). **Appraising Success of LLM-based Dialogue Agents**. *INDIACom 2024*. IEEE.

Zhou, J., Jiang, B., Gong, R., & Jiang, H. (2025). **Evaluating LLM-based Role-playing Agent Authenticity via Event-driven Response Simulation**. *ISAICS 2025*. IEEE. DOI: 10.1109/ISAICS66888.2025.11350200.